lesson11_practice_solution.py Explanation

Softmax 함수 살펴보기

Softmax 함수는 머신러닝, 특히 **다중 클래스 분류(Multi-class Classification)** 문제에서 자주 사용되는 활성화 함수이다.

1. Softmax의 역할

여러 개의 점수(logit)를 입력받아 이를 **확률 분포**로 변환한다.

여기서 점수는 분류를 위해서 컴퓨터가 계산한 점수이다. 이 점수를 비교하기 쉬운 비율로 바꿔주는 역할을 한다.

예를 들어, 신경망이 어떤 이미지에 대해 다음과 같은 점수를 출력했다고 가정해보자.

$$z = [2.0, 1.0, 0.1]$$

이 값들은 단순 점수일 뿐 확률이 아니다. Softmax를 적용하면:

$$[0.659, 0.242, 0.099]$$

처럼 변환된다.

특징:

- 모든 값이 0 ~ 1 사이
- 전체 합이 1
- 각 값을 확률로 해석 가능

2. 수식

입력 벡터 $z = [z_1, z_2, \dots, z_n]$에 대해

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

이다.

즉,

1. 각 원소에 지수 함수 e^x를 적용

2. 모든 지수값의 합으로 나눔

3. 계산 예시

입력:

$$[2.0, 1.0, 0.1]$$

Step 1. 지수 계산

$$e^2 = 7.389$$

$$e^1 = 2.718$$

$$e^{0.1} = 1.105$$

Step 2. 합 계산

$$7.389 + 2.718 + 1.105 = 11.212$$

Step 3. 정규화

$$\frac{7.389}{11.212} = 0.659$$

$$\frac{2.718}{11.212} = 0.242$$

$$\frac{1.105}{11.212} = 0.099$$

결과:

$$[0.659, 0.242, 0.099]$$

4. 왜 지수 함수(exp)를 사용할까?

지수 함수는 큰 값을 더욱 크게 만들고 작은 값을 더욱 작게 만든다.

예:

입력:

$$[3, 2, 1]$$

지수 적용:

$$[e^3, e^2, e^1] = [20.1, 7.4, 2.7]$$

원래 차이는 1씩이었지만 지수 함수 적용 후 차이가 크게 벌어진다.

덕분에 모델이 가장 가능성이 높은 클래스를 더 명확하게 선택할 수 있다.

5. 실제 사용 예

고양이/개/토끼 분류 모델의 출력:

클래스	Logit
고양이	4.2
개	1.8
토끼	0,5

Softmax 적용 후:

클래스	확률
고양이	0.89
개	0.09
토끼	0.02

따라서 모델은:

> "89% 확률로 고양이다."

라고 해석할 수 있다.

6. 수치적 안정성 (Numerical Stability)

실제로는 다음과 같이 계산한다.

$$Softmax(z_i) = \frac{e^{z_i - max(z)}}{\sum^{e^{Z_j - max(z)}}}$$

예를 들어:

$$[1000, 1001, 1002]$$

에 대해 e^{1002}는 너무 커서 오버플로우가 발생할 수 있다.

그래서 최대값(1002)를 빼서

$$[-2, -1, 0]$$

으로 만든 뒤 계산합니다.

확률 결과는 동일하지만 훨씬 안정적이다.

7. Sigmoid와의 차이

항목	Sigmoid	Softmax
출력 개수	각 노드 독립	전체 노드 연관
범위	0 ~ 1	0 ~ 1
합	합이 1이 아님	합이 항상 1
주 용도	이진 분류	다중 클래스 분류

예:

Sigmoid

$$[0.8, 0.7, 0.6]$$

$\rightarrow$ 각 클래스가 독립

Softmax

$$[0.8, 0.15, 0.05]$$

→ 클래스들이 서로 경쟁

8. 직관적으로 이해하기

Softmax는

> "점수(score)를 확률(probability)로 변환하는 함수"

라고 생각하면 됩니다.

입력:

$$[2.6, 0.3, -1.2]$$

$$\downarrow$$

$$Softmax$$

$$\downarrow$$

$$[0.86, 0.12, 0.02]$$

즉, 모델의 원시 출력(logit)을 "전체 합이 1인 확률 분포"로 바꾸어 주는 것이 Softmax의 핵심 역할이다.

---

lesson11_practice_solution.py 코드 설명

```
"""Lesson 11 실습: Self-Attention 과 Transformer 직접 구현해 보기
======================================================================
실행:  python lesson11_practice.py          (전체 실행)
       python lesson11_practice.py 1 3      (Part 1, 3 만 실행)
필요:  numpy, matplotlib, torch (Part 6~7)

사용 방법
  * '# TODO' 로 표시된 곳을 채우세요. 각 Part 의 check(...) 가 모두 PASS 면 정답입니다.
  * 막히면 lesson11_practice_solution.py 를 참고하세요.
  * Part 1~5: NumPy 로 수식을 그대로 구현 -> Part 6~7: PyTorch 로 Transformer 블록 구성/학습
```

```
"""
import sys
import numpy as np
import matplotlib.pyplot as plt
```

```
rng = np.random.default_rng(0)
```

NumPy 난수 생성기(Random Number Generator)를 만들고, 시드(seed)를 고정하여 재현 가능하게 한다. 항상 동일한 난수를 생성하도록 하기 위해서 시드를 0으로 고정한다.

```
def check(name, ok):
    print(f"  [{'PASS' if ok else 'FAIL'}] {name}")
    return ok
```

- `ok`는 조건
- `name`은 `ok`에 대한 설명

어떤 조건이 성공인지 실패인지에 대한 출력을 제어하는 함수

---

softmax()

```
def softmax(x, axis=-1):
    """수치 안정성을 위해 최댓값을 빼고 계산한다 (exp 오버플로 방지)."""
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)

    return e / e.sum(axis=axis, keepdims=True)
```

`softmax`의 결과의 차원은 `x`의 `shape`와 동일하다.

```
x = x - x.max(axis=axis, keepdims=True)
```

`axis`는 어떤 축을 기준으로 계산할 지를 지정한다. 만일 2차원 배열에서 0이면 행을 따라가면서 그 아래에 있는 각 열에서 `max`값을 찾는다. 반면, `axis=1`이면 열을 따라 이동하면 `max`값을 찾는다. 그렇기 때문에, 0열에 있는 행에서 `max`값을 찾고, 다음 열 옆에 있는 행에서 `max`값을 찾는다.

```
x = np.array([
    [1, 5, 3],
    [4, 2, 8]
])
x.max(axis=0)   # 행을 따라가며 각 열에서 최댓값을 구함. 결과: [4, 5, 8]
x.max(axis=1)   # 열을 따라가며 각 행에서 최댓값을 구함. 결과: [5, 8]
x.max(axis=-1)  # 마지막 차원을 의미하기 때문에 axis = 1과 동일. 결과: [5, 8]
```

`keepdims=True`는 차원을 보존하라는 의미이다. 원래 배열이 2차원이었기 때문에 `max`의 결과는:

```
[[5]
 [8]]
```

`np.exp(x)`는 e^x로 양수, 음수, 그리고 벡터도 `x`로 입력 가능하다.

```
x = np.array([
    [1, 5, 3],
    [4, 2, 8]
])

xs = x.sum(axis=1, keepdims=True)
print(xs)
```

`x.sum()`의 결과는

```
[[ 9]
 [14]]
```

아래 코드를 실행한 결과는

```
xs = x.sum(axis=0, keepdims=True)
print(xs)
```

다음과 같다.

```
[[ 5  7 11]]
```

```python
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a @ b
```

결과:

$$1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

행렬끼리 사용

```python
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

A @ B
```

즉, `A @ B`는

$$\begin{bmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

```python
scores = np.where(mask, scores, -1e9)
```

`.where` 메서드는 조건에 따라 값을 선택하는 함수이다.

> `np.where(조건, 참일 때 값, 거짓일 때 값)`

즉, `조건 ? 참일 때 값 : 거짓일 때 값`과 비슷한 역할을 한다.

`def attention(Q, K, V, mask=None, scale=True):`

```python
def attention(Q, K, V, mask=None, scale=True):
    """Q,K: (T, d_k)   V: (T, d_v)   mask: (T, T) bool, True = 볼 수 있음.
    return: (출력 Z (T, d_v), 어텐션 가중치 A (T, T))"""
    d_k = Q.shape[-1]
    scores = Q @ K.T                        # (T, T)  모든 (i, j) 쌍의 내적

    if scale:
        scores = scores / np.sqrt(d_k)    # 식 11.4
    if mask is not None:
        scores = np.where(mask, scores, -1e9)   # 못 보는 위치는 -inf 에 가깝게

    A = softmax(scores, axis=-1)          # 각 행(질의)마다 키 방향으로 softmax

    return A @ V, A
```

이 함수는 Scaled Dot-Product Attention을 구현한 코드이다. Transformer에서 사용하는 핵심 어텐션 메커니즘과 거의 동일한 형태이다.

먼저 전체 흐름부터 보면:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

이 수식을 그대로 코드로 옮긴 것이다.

1. 함수 정의

입력:

- Q (Query): (T, d_k)
- K (Key): (T, d_k)
- V (Value): (T, d_v)

여기서

- T: 토큰 개수 (시퀀스 길이)

- d_k: Query와 Key의 차원
- d_v: Value의 차원

예를 들어 문장이

```
[I, love, AI]
```

라면

```
T = 3
```

이고 각 단어가 64차원 벡터라면

```
Q.shape = (3, 64)
K.shape = (3, 64)
```

이 된다.

2. Key 차원 구하기

```
d_k = Q.shape[-1]
```

마지막 차원 크기, 즉 Key/Query 벡터의 길이를 구한다.

예:

```
Q.shape = (5, 64)
```

이면

```
d_k = 64
```

추가 설명

`.shape`는 배열의 각 차원의 크기를 담고 있는 튜플(tuple)을 반환한다.

```
x = np.zeros((10, 20, 30))
print(x.shape)
```

결과:

> (10, 20, 30)

즉,

- 첫 번째 차원 크기: 10
- 두 번째 차원 크기: 20
- 세 번째 차원 크기: 30

이다.

```
d_k = Q.shape[-1]
```

`Q.shape`는 `(T, d_k)`이며, `T`는 시퀀스 길이, `d_k`는 키 벡터의 차원 수를 의미한다.

그러므로, `Q.shape[-1]`은 `d_k`이다.

3. Attention Score 계산

```
scores = Q @ K.T
```

핵심 부분이다.

차원

```
Q       : (T, d_k)
K.T     : (d_k, T)
```

행렬곱 결과

```
scores: (T, T)
```

의미

각 원소

$$scores_{ij} = Q_i \cdot K_j$$

즉,

> i번째 Query가 j번째 Key와 얼마나 관련 있는가?

를 나타낸다.

예를 들어

```
Q =
[[1, 0],
 [0, 1]]

K =
 [[1, 0],
  [1, 1]]
```

이면

```
Q @ K.T
=
[[1, 1],
 [0, 1]]
```

즉,

```
1번 단어는
- 자기 자신과 1
- 2번 단어와도 1

2번 단어는
- 1번 단어와 0
- 2번 단어와 1
```

의 유사도를 가진다.

4. Scaled Attention

```
if scale:
    scores = scores / np.sqrt(d_k)
```

Transformer 논문에서 사용한 스케일링이다.

수식:

$$\frac{QK^T}{\sqrt{d_k}}$$

왜 필요한가?

차원이 커질수록 내적값이 너무 커진다.

예:

```
d_k = 512
```

이면

```
Q · K = 매우 큰 값
```

이 될 수 있다.

그 상태에서 Softmax를 하면

```
[100, 2, 1]
```

같은 값이 나와

```
softmax
=
[0.99999, 0.000001, 0]
```

처럼 거의 원-핫 벡터가 된다.

그러면,

- 학습이 불안정해지고
- Gradient가 작아지는 문제

가 발생한다.

따라서

$$\sqrt{512} \quad \approx \quad 22.6$$

으로 나누어 값을 적당한 범위로 유지한다.

5. Mask 적용

```
if mask is not None:
    scores = np.where(mask, scores, -1e9)
```

역할

특정 위치를 보지 못하게 만든다.

예를 들어

```
scores =
[[2, 3, 4],
 [1, 2, 3],
 [5, 1, 2]]
```

이고

결과는

```
[[2, -1e9, -1e9],
 [1, 2, -1e9],
 [5, 1, 2]]
```

이 된다.

왜 -1e9인가?

Softmax에 넣으면

$$e^{-10^9} \approx 0$$

이므로

```
softmax([2, -1e9, -1e9])
```

는

```
[1, 0, 0]
```

과 거의 같다.

즉

> 해당 위치는 절대 참조하지 않음

을 의미한다.

Note

Softmax 함수란?

Softmax 함수는 '**점수(score)들을 확률로 바꿔주는 함수**'이다.

예를 들어 점수가 [2, 1, 0]이면,

1번 후보가 가장 좋음

2번 후보가 그 다음

3번 후보가 가장 낮음

정도만 알 수 있다.

소프트맥스를 적용하면,

```
x = [2, 1, 0]
m = max(x)  # m == 2
x = x - m   # x = [0, -1, -2]
e = np.exp(x)   # e = [1.   0.36787944 0.13555528]
e /= e.sum(axis=0, keepdims=True)   # e.sum = [1.50321472], e =
[0.66524096 0.24472847 0.09003057]
```

즉

```
[0.665, 0.245, 0.090]
```

이 된다. 이제 뭐가 단순히 좋다가 아니라,

```
token 1: 66.5%
token 2: 24.5%
token 3: 9.0%
```

로 해석할 수 있다.

> 특징은
>
> 1. 모든 값이 0~1사이
> 2. 전체 합이 1
> 3. 원래 점수가 클수록 더 큰 확률
>
> 이다.
>
> **Attention**에서의 역할은
>
> ```
> scores = Q @ K.T
> ```
>
> 로 얻은"유사도 점수(scores)"를
>
> ```
> A = softmax(scores)
> ```
>
> 를 통해 **'각 토큰을 얼마나 참고할지'**를 나타내는 **Attention 가중치(확률)**로 바꾸는 것이다.

```
A @ V
```

"Attention이 중요하다고 판단한 토큰들의 정보를 비율(A)에 따라 섞어서 만든 새로운 문맥 표현 (Context Vector)"이다.

자연어로 생각해보기

문장:

```
철수가 사과를 먹었다.
```

"먹었다"를 처리한다고 가정해보자.

Attention 결과가

```
A = [0.6, 0.3, 0.1]
```

이라면

```
철수: 60%
사과: 30%
먹었다: 10%
```

를 참고하라는 의미이다.

그러면

```
Z
=
0.6 * V(철수)
+ 0.3 * v(사과)
+ 0.1 * V(먹었다)
```

가 된다.

즉, 이 결과는 "먹었다"가 바라본 문맥 정보이다. 그러므로, "철수"와 가장 관계가 깊다.

왜 Value를 사용하나?

다음 궁금증을 가질 수 있다.

```
Query와 Key로 유사도를 구했는데
왜 마지막에는 Value를 사용하지?
```

생각해보면

```
Query = 무엇을 찾고 싶은가
Key = 내가 어떤 정보인지
Value = 실제 전달할 정보
```

이다.

도서관 비유로 보면

```
Query = 찾고 싶은 책 제목
Key = 책장의 분류표
Value = 실제 책 내용
```

이다.

책을 찾을 때는 $Query \leftrightarrow Key$ 비교를 하지만, 실제로 가져오는 것은 $Value$이다. Attention도 동일하다.

`def part1():`

```
def part1():
    print("Part 1. scaled dot-product attention")
    x = np.array([[1.0, 2.0, 3.0], [1000.0, 1001.0, 1002.0]])
    s = softmax(x)
    check("softmax 행의 합 = 1, 큰 값에서도 NaN 없음", np.allclose(s.sum(1), 1) and not np.isnan(s).any())
    check("softmax 는 평행이동에 불변", np.allclose(s[0], s[1]))
    Q = K = np.eye(3)
    V = np.array([[1.0, 0], [0, 1.0], [5.0, 5.0]])
    Z, A = attention(Q, K, V)
    check("A 의 각 행 합 = 1", np.allclose(A.sum(1), 1))
    check("Z = A @ V", np.allclose(Z, A @ V))
    check("대각 성분이 가장 큼 (자기 자신과 가장 유사)", (A.argmax(1) == np.arange(3)).all())
```

- `np.allclose()`: **"두 배열이 거의 같은지를 확인하는 함수"**
  - 0.1 + 0.2 == 0.3이 실수 오차 때문에 `False`인 경우가 있다.
  - 이런 오차를 막기 위해서 `np.allclose()`를 사용한다.

```
np.allclose(
    0.1 + 0.2,
    0.3
)   # True이다.
```

  `allclose()`는 아주 작은 오차는 무시한다.
- `np.eye(3)`: 3행 3열인 단위 행렬을 만드는 함수

`.argmax()`는 배열에서 가장 큰 값의 위치(인덱스)를 반환하는 함수이다.

이차원 배열의 예:

```
A = np.array([
    [1, 5, 2],
    [7, 3, 4]
])

A.argmax()
```

답은 **3**이다.

NumPy 배열은

```
[1, 5, 2, 7, 3, 4]
```

처럼 펼쳐서 생각한다.

axis를 사용할 경우:

```
A = np.array([
    [1, 5, 2],
    [7, 3, 4]
])

A.argmax(axis=1)
```

결과는

```
array([1, 0])
```

각 행에서 가장 큰 숫자의 인덱스를 반환한다.

```
A = np.array([
    [1, 5, 2],
    [7, 3, 4]
])

A.argmax(axis=0)
```

결과는

```
array([1, 0, 1])
```

`np.arange()`는 **일정한 간격으로 숫자를 생성하는 함수**다.

Python의 `range()`와 비슷하지만 NumPy 배열을 반환한다.

`np.arange(3)`의 결과는 `array([0, 1, 2])`이다.

```
np.arange(start, stop, step)
```

`A.argmax(1) == np.arange(3).all()`의 해석

1. A.argmax(1)

여기서 A는 Attention Weight 행렬이다.

예를 들어

```
A =
[
    [0.9, 0.1, 0.0],
    [0.2, 0.7, 0.1],
    [0.1, 0.2, 0.7]
]
```

이면

```
A.argmax(axis=1)
```

은 각 행에서 최대값의 위치를 찾는다.

결과는

```
array([0, 1, 2])
```

이다.

즉

```
1행은 0열에 가장 집중
2행은 1열에 가장 집중
3행은 2열에 가장 집중
```

이라는 의미이다.

2. np.arange(3)

```
np.arange(3)
```

결과:

```
array([0, 1, 2])
```

이다.

3. 비교

```
A.argmax(1) == np.arange(3)
```

이므로

```
[0, 1, 2] == [0, 1, 2]
```

결과는

```
array([True, True, True])
```

4. .all()

```
[True, True, True].all()
```

의 결과는

```
True
```

이다.

Attention 관점에서의 의미

결국

```
(A.argmax(1) == np.arange(3)).all()
```

은

> "각 토큰이 가장 강하게 주목(attend)하는 대상이 자기 자신인가?"

를 검사하는 코드이다.

`def part2()`

```
def part2():
    print("Part 2. self-attention 층 + 히트맵 (그림 11.1 따라 하기)")
    T, d, dk = len(WORDS), 16, 8
    X = rng.normal(size=(T, d))
    Wq, Wk, Wv = (rng.normal(size=(d, dk)) / np.sqrt(d) for _ in range(3))
    Z, A = self_attention(X, Wq, Wk, Wv)
    check("출력 shape = (T, d_k)", Z.shape == (T, dk))
    check("A shape = (T, T), 행 합 = 1", A.shape == (T, T) and
np.allclose(A.sum(1), 1))

    # 의미 있는 어텐션을 '손으로' 만들어 보기: brilliant(마지막 행)가 solution(1번
열)을 보게 한다.
    X2 = rng.normal(size=(T, d)) * 0.3
    X2[1, 0] = 3.0        # solution : '주어' 표식 (0번 차원)
    X2[-1, 1] = 3.0       # brilliant: '수식어' 표식 (1번 차원) -> 주어를 찾는다
    Wq2 = np.zeros((d, dk)); Wq2[1, 0] = 3.0   # 수식어 표식 -> 질의 0 번 성분
    Wk2 = np.zeros((d, dk)); Wk2[0, 0] = 3.0   # 주어 표식   -> 키 0 번 성분

    _, A2 = self_attention(X2, Wq2, Wk2, np.eye(d)[:, :dk])
    check("brilliant 가 solution 에 가장 큰 가중치", A2[-1].argmax() == 1)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))

    for a, M, t in zip(ax, (A, A2), ("random weights", "hand-designed
weights")):
        a.imshow(M, cmap="Blues"); a.set_title(t)
        a.set_xticks(range(T), WORDS, rotation=45, ha="right");
a.set_yticks(range(T), WORDS)

    plt.tight_layout(); plt.savefig("part2_attention_map.png", dpi=120);
plt.close()
    print("  -> part2_attention_map.png 저장 (행=질의, 열=키)")
```

```
T, d, dk = len(words), 16, 8
```

1. T: 토큰(Token)의 개수

예를 들어

```
WORDS = ["The", "Solution", "to", "the", "problem", "was", "brilliant"]
```

이면

```
T = 7
```

이다.

즉,

- 문장 안의 단어 개수
- 시퀀스 길이(sequence length)

를 의미한다.

어텐션 행렬 A의 크기가

```
A.shape == (T, T)
```

인 이유도,

- 행: 질의(Query) 토큰
- 열: 키(Key) 토큰

이기 때문이다.

예를 들어 T = 7이면

> The solution to ...
>
> The
>
> solution
>
> to
>
> ...

처럼 7×7 행렬이 된다.

2. d: 단어 임베딩 차원

```
d = 16
```

각 단어를 길이 16짜리 벡터로 표현한다는 뜻이다.

```
X.shape = (T, d)
```

즉

```
X = rng.normal(size=(T, d))
```

라면

```
X =
[
    word1(16차원),
    word2(16차원),
    ...
]
```

형태이다.

예를 들어

```
solution → [0.2, -1.1, 0.4, ..., 0.7]
```

처럼 길이 16의 벡터 하나로 표현된다.

Note

임베딩(embedding)

> 임베딩은 **"단어나 토큰을 숫자 벡터로 바꾼 것"**이다.

임베딩이 필요한 이유

컴퓨터는

```
solution
brilliant
problem
```

같은 단어를 그대로 이해하지 못한다.

그래서 먼저 숫자로 바꿔야 한다. 가장 단순한 방법은 번호를 붙이는 것이다.

```
solution  -> 0
problem   -> 1
brilliant -> 2
```

하지만 이렇게 하면 문제가 있다.

컴퓨터 입장에서는

```
2 > 1 > 0
```

이라는 숫자 관계 밖에 보이지 않는다. 즉, "solution"과 "problem"이 의미상 관련 있다는 정보는 전혀 없다.

임베딩 벡터

그래서 단어를 하나의 숫자가 아니라 여러 숫자의 묶음으로 표현한다.

예를 들어

```
solution
```

을

```
[0.8, -0.3, 1.2, 0.4]
```

처럼 나타낸다.

그리고

```
answer
```

는

```
[0.9, -0.2, 1.1, 0.5]
```

처럼 빗슷한 벡터를 갖도록 학습된다.

즉,

solution ≈ answer

라는 의미적 관계가 벡터 공간에 저장된다.

코드에서의 임베딩

`part1()`에서는

```
d = 16
x = rng.normal(size=(T, d))
```

라고 되어있다.

여기서

```
d = 16
```

은 "각 단어를 길이 16짜리 벡터로 표현한다"는 뜻이다.

예를 들어 단어가 5개라면

```
X.shape
```

는

```
(5, 16)
```

이다.

실제로는

```
X =
[
    [ 0.1,  0.2, ..., 1.3],  # word1
    [-0.3,  0.8, ..., 0.5],  # word2
    [ 1.2, -0.4, ..., 0.1],  # word3
    ...
]
```

처럼 생겼다.

행 하나가 단어 하나의 임베딩이다.

Self-Attention 관점에서 보면

문장

```
The solution to the problem was brilliant
```

가 있으면, 먼자 각 단어가 임베딩으로 변환된다.

```
"The"       → 16차원 벡터
"solution"  → 16차원 벡터
"to"        → 16차원 벡터
"the"       → 16차원 벡터
"problem"   → 16차원 벡터
"was"       → 16차원 벡터
"brilliant" → 16차원 벡터
```

그래서

```
X.shape = (7, 16)
```

이 된다.

그 다음에 Self-Attention이

```
Q = XWq
K = XWk
V = XWv
```

를 계산하여 각 단어가 다른 단어를 얼마나 참고해야 하는지 찾는다.

---

3. dk: Query/Key/Value의 차원

```
dk = 8
```

Self-Attention에서는 원래 임베딩을 그대로 사용하지 않고

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

로 투영(projection) 한다.

그래서

```
wq.shape = (d, dk)
wk.shape = (d, dk)
wv.shape = (d, dk)
```

즉,

```
(16, 8)
```

크기이다.

결과적으로

```
Q.shape = (T, dk)
K.shape = (T, dk)
V.shape = (T, dk)
```

가 되어

```
(T, 8)
```

이 된다.

전체 Shape 흐름

현재 코드에서는

```
T = len(WORDS)
d = 16
dk = 8
```

이므로

```
입력 X        : (T, 16)

Wq, Wk, Wv    : (16, 8)

Q = XWq       : (T, 8)
K = XWk       : (T, 8)
V = XWv       : (T, 8)

A             : (T, T)
Z             : (T, 8)
```

가 된다.

그래서 이 검사가 통과한다.

```
check("출력 shape = (T, d_k)", Z.shape == (T, dk))
```

왜 dk를 따로 두나?

민약 원래 임베딩 차원 `d=768`인 모델이라면,

어텐션을 계산할 때도 768차원을 그대로 쓰면 계산량이 많다.

그래서 보통

```
d = 768
dk = 64
```

처럼 더 작은 공간으로 투영한 뒤

$$QK^T$$

를 계산한다.

Transformer의 멀티헤드 어텐션에서도 각 헤드가 보통 이런 작은 `d_k`를 사용한다.

요약하면:

- T: 단어(토큰) 수 = 시퀀스 길이
- d: 입력 임베딩 차원
- dk: Query/Key/Value 벡터 차원
- A.shape = (T, T): 각 단어가 다른 단어를 얼마나 보는지
- Z.shape = (T, dk): 어텐션 결과 출력 벡터들이다.

투영(projection)의 의미

> "원래 벡터를 다른 좌표계로 바꾸는 것"

코드에서

```
Q = X @ Wq
```

가 바로 투영이다.

예를 들어 단어 "brilliant"가

```
x = [2.1, 0.7, -1.3, ...]
```

이라는 16차원 벡터라면

```
q = x @ Wq
```

로 계산하면

```
q = [1.2, -0.5, ...]
```

라는 8차원 벡터가 된다. 이 과정이 투영이다.

왜 굳이 투영할까?

원래 임베딩에는 엄청 많은 정보가 들어있다.

예를 들어

```
brilliant
```

에 대한 임베딩은

- 형용사인가?
- 긍정적 의미인가?
- 사름을 수식하는가?
- 문제를 수식하는가?
- 학문 관련이가?

등등의 정보가 섞여 있다.

하지만 어텐션은

> "누를 봐야 하는가?"

만 판단하면 된다.

그래서

```
Q = XWq
```

를 통해

```
어텐션 계산에 필요한 특징만 추출
```

한다.

마치 사진을 흑백으로 바꾸는 것과 비슷하다.

원본:

```
RGB 3채널
```

어텐션용:

```
흑백 1채널
```

정보는 줄지만 필요한 특징은 남는다.

이 코드에서 손으로 만든 예

아래 코드에서

```
X2[1, 0] = 3.0      # solution
X2[-1, 1] = 3.0     # brilliant
```

여기서는

- 0번 축 = "주어 표시"
- 1번 축 = "수식어 표식"

처럼 인위적으로 의미를 넣었다.

그리고

```
Wq2[1, 0] = 3.0
```

은

$$\begin{matrix}\text{수식어 특징}\\ \downarrow \\ \text{Query의 0번 성분}\end{matrix}$$

으로 매핑한다.

반대로

```
wk2[0, 0] = 3.0
```

은

$$\begin{matrix}\text{주어 특징}\\ \downarrow \\ \text{Key의 0번 성분}\end{matrix}$$

으로 매핑한다.

즉, 투영 행렬 `Wq2`, `Wk2`가

> "어떤 특징을 어텐션 계산에 사용할지"

선택해 주는 역할을 한다.

선형대수 관점

사실 투영은 단순히

$$q = xW_q$$

라는 행렬곱이다.

예를 들어

$$x = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

$$W_q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

이면

$$\begin{aligned} x \cdot W_q = q = \begin{bmatrix} 1\times1+2\times0+3\times1 & 1\times0+2\times1+3\times1 \end{bmatrix} \\ = \begin{bmatrix} 4 & 5 \end{bmatrix} \end{aligned}$$

가 된다.

원래 3차원 벡터가 2차원 벡터로 바뀌었다.

이 과정이 차원 축소이자 투영이다.

Transformer 처리 과정

$$
\begin{gathered}
\text{임베딩 X} \\
\downarrow \\
Wq \\
\downarrow \\
\text{Query(무엇을 찾고 싶은가?)}
\end{gathered}
$$

$$
\begin{gathered}
\text{임베딩 X} \\
\downarrow \\
Wk \\
\downarrow \\
\text{Key(무슨 정보를 가지고 있나?)}
\end{gathered}
$$

$$
\begin{gathered}
\text{임베딩 X} \\
\downarrow \\
Wv \\
\downarrow \\
\text{Query(실제로 전달할 정보)}
\end{gathered}
$$

`X2 = rng.normal(size=(T, d)) * 0.3`

`rng`는 NumPy의 난수 생성기이다.

`normal()`은 평균 0, 표준편차 1인 표준정규분포 `N(0, 1)`에서 난수를 생성한다.

다음 예의

```
X = rng.normal(size=(3, 2))
print(X)
```

결과는

```
[[ 0.12573022 -0.13210486]
 [ 0.64042265  0.10490012]
 [-0.53566937  0.36159505]]
```

이다.

즉 3행 2열의 행렬에 -1 ~ 1 사이의 정규분포를 갖는 값을 생성한다.

`Wq2 = np.zeros((d, dk))`

모든 값이 0인 `d`행, `dk`열인 행렬을 만든다.

Transformer의 Attention이 특정 토큰을 찾도록 설정

```
X2[1, 0] = 3.0        # solution : '주어' 표식 (0번 차원)
X2[-1, 1] = 3.0       # brilliant: '수식어' 표식 (1번 차원) -> 주어를 찾는다
Wq2 = np.zeros((d, dk)); Wq2[1, 0] = 3.0   # 수식어 표식 -> 질의 0 번 성분
Wk2 = np.zeros((d, dk)); Wk2[0, 0] = 3.0   # 주어 표식   -> 키 0 번 성분
```

Attention이 특정 토큰을 찾도록 인위적으로 설계한 예제이다.

> **Note**
>
> ### Self-attention
>
> 같은 입력 `X`에서 세 가지 다른 "관점"으로 변환해 만드는 것입니다.
>
> `Q = X @ Wq` (질의: "나는 무엇을 찾고 있나?")
> `K = X @ Wk` (키: "나는 어떤 정보로 찾아질 수 있나?")
> `V = X @ Wv` (값: "찾아지면 내가 넘겨줄 내용")
>
> - 입력 `X`는 하나입니다. 달라지는 것은 곱하는 가중치 행렬 `Wq`, `Wk`, `Wv`이다.
> - 이 세 행렬이 학습되는 파라미터이다. `Q`, `K`, `V` 자체는 `X`가 바뀔 때마다 새로 계산되는 값입니다.
> - **"self"는 Q, K, V가 모두 같은 시퀀스에서 나온다는 뜻**이다. 대비되는 cross-attention은 `Q`는 디코더에서, `K`와 `V`는 인코더에서 가져온다.
> - 같은 단어 벡터에서 출발해도 `Wq`, `Wk`, `Wv`가 다르기 때문에 역할이 갈린다. 한 토큰이 질의자(`Q`)이면서 동시에 피질의자(`K`)이고, 정보 제공자(`V`)이기도 하다.
>
> 계산 흐름은 다음과 같다.
>
> 1. `Q @ K.T`로 모든 토큰 쌍 (i, j)의 유사도를 구한다. "i번 토큰의 질의가 j번 토큰의 키와 얼마나 맞나?"
> 2. softmax로 행마다 합이 1인 가중치 `A`를 만든다.
> 3. `A @ V`로 각 토큰의 출력 = 다른 토큰들의 `V`를 가중합한 값이 된다.

"brilliant가 solution을 본다"는 관계를 만드는 방법

핵심은 내적 `q_brilliant · k_solution`만 크게 만드는 것이다. 코드의 각 줄은 그 목적을 위한 장치이다.

문장은 **The(0) solution(1) that(2) the(3) professor(4) recommended(5) was(6) brilliant(7→ 인덱스 -1)**로, `T=7` 단어 기준 마지막이 `brilliant`이다.

단계 1: 입력에 "표식"을 심는다.

```
X2 = rng.normal(size=(T, d)) * 0.3   # 작은 잡음 (배경)
X2[1, 0]  = 3.0   # solution의 0번 차원 = "나는 주어다"
X2[-1, 1] = 3.0   # brilliant의 1번 차원 = "나는 수식어다"
```

임베딩 16개 차원 중 0번과 1번 차원에 의미를 임의로 부여한다. 이 의미는 여기서 그렇게 하겠다는 것이지 컴퓨터는 **주어**, **수식어**를 알지 못한다. 나머지는 잡음이라 값이 작다(약 0.3 수준).

단계 2: Wq가 "수식어 표식"을 질의로 바꾼다.

```
Wq2[1, 0] = 3.0
```

`Q = X2 @ Wq2`에서 `Q`의 0번 성분은 `X2[:, 1] * 3.0`입니다. 즉 입력의 1번 차원(수식어 표식)이 클수록 질의 0번 성분이 커진다.

- `brilliant: 3.0 × 3.0 = 9.0`
- 다른 토큰: 약 0.3 × 3.0 ≈ 작은 값

의미는 "수식어인 토큰은 질의 0번 방향으로 주어를 찾는다"이다.

단계 3: Wk가 "주어 표식"을 키로 바꾼다.

```
Wk2[0, 0] = 3.0
```

`K`의 0번 성분은 `X2[:, 0] * 3.0` 이다.

- solution: 3.0 × 3.0 = 9.0
- 다른 토큰: 작은 값

의미는 "주어인 토큰은 키 0번 방향에 신호를 낸다"이다.

단계 4: 내적이 두 신호를 만나게 한다

`q_i · k_j`에서 0번 성분끼리의 곱이 지배적입니다.

(질의, 키)	0번 성분 곱	크기
brilliant → solution	9 × 9 = 81	매우 큼
brilliant → 다른 단어	9 × (작은 값)	작음
다른 단어 → solution	(작은 값) × 9	작음

($\sqrt{d_k}$로 나눈 뒤에도 이 크기 차이는 그대로 유지된다.)

softmax는 큰 값을 크게 키우므로, brilliant 행에서 solution 열이 거의 1에 가까워집진다.

단계 5: Wv는 건드리지 않는다

`np.eye(d)[:, :dk]`는 입력을 그대로 통과시키는 투영이다. 이 실험은 어텐션 가중치 `A`가 어디를 향하는지만 보려는 것이라, `V`는 단순하게 둔다.

직관 요약

```
brilliant  --(Wq: 수식어면 '주어를 찾아라' 질의 생성)--> q 크게
solution   --(Wk: 주어면 '여기 있다' 키 생성)---------> k 크게
                     q · k  =  큰 값  --softmax--> 가중치 1
```

`Q`와 `K`가 같은 차원(0번 성분)에서 서로 맞아떨어지도록 가중치를 설계한 것이 전부이다. 같은 원리가 실제 Transformer에서는 데이터로부터 학습된다. 학습된 `Wq`, `Wk`가 "형용사 → 수식 대상 명사", "대명사 → 지시 대상" 같은 관계를 스스로 만들어 냅니다. 이 코드는 그 학습 결과를 손으로 흉내 낸 미니어처이다.

`Q`와 `K`가 같은 차원(0번 성분)에서 서로 맞아떨어지도록 가중치를 설계한 것이 전부이다. 같은 원리가 실제 Transformer에서는 데이터로부터 학습됩니다. 학습된 `Wq`, `Wk`가 "형용사 → 수식 대상 명사", "대명사 → 지시 대상" 같은 관계를 스스로 만들어 낸다. 이 코드는 그 학습 결과를 손으로 흉내 낸 미니어처이다.

Part 3

```
# Part 3. 왜 sqrt(d_k) 로 나눌까?
#   q, k 의 각 성분이 평균 0, 분산 1 이면 q.k 의 분산은 d_k 이다.
#   스케일을 안 하면 d_k 가 클수록 softmax 가 one-hot 처럼 뾰족해져 기울기가 사라진다.
```

```
def softmax_peakiness(dk, scale, T=16, trials=200):
    """softmax 가중치의 평균 최댓값 (1/T: 완전히 평평, 1: one-hot)."""
    vals = []
    for _ in range(trials):
        Q = rng.normal(size=(T, dk)); K = rng.normal(size=(T, dk))
        _, A = attention(Q, K, np.zeros((T, 1)), scale=scale)
        vals.append(A.max(axis=1).mean())

    return float(np.mean(vals))
```

softmax 가중치가 얼마나 뾰족한지(한 곳에 쏠려 있는지)를 숫자 하나로 재는 함수이다. 이 함수는 `part3()`에서 "왜 $\sqrt{d_k}$"로 나누는지를 살펴보는데 사용된다.

`for`문의 동작

1. 랜덤 `Q`, `K`로 어텐션 가중치 `A(T×T)`를 만든다. `V`는 쓰지 않으므로 `np.zeros`로 채운다.
2. `A.max(axis=1)`로 각 행(질의)에서 가장 큰 가중치를 뽑고, `.mean()`으로 행들의 평균을 낸다.
3. 이를 200번 반복해 평균낸다. 난수 하나에 결과가 좌우되지 않게 하려는 것이다.

반환값의 의미

값	의미
1/T (T=16이면 0.0625)	모든 위치에 균등하게 분산됨 (완전히 평평)
1	한 위치에 100% 쏠림 (one-hot)

값이 클수록 softmax가 뾰족하다.

무엇을 보여 주나

`q`, `k`의 성분이 평균 0, 분산 1이면 `q·k`의 분산은 `d_k`입니다. 표준편차는 $\sqrt{d_k}$이다.

- `scale=False: d_k`가 커질수록 점수의 편차가 커진다. softmax는 큰 값 차이를 증폭하므로 한 위치가 거의 1을 차지하고, 반환값이 1에 가까워진다.
- `scale=True: √d_k`로 나누면 점수의 분산이 `d_k`와 상관없이 약 1로 유지된다. 반환값이 `d_k`에 관계없이 비슷한 중간 수준으로 남는다.

왜 문제인가

softmax가 one-hot에 가까워지면 포화(saturation)가 일어난다. 출력이 거의 0 또는 1이라 기울기가 거의 0이 되고, 역전파 때 학습 신호가 사라진다. softmax_peakiness는 이 현상을 `d_k`를 키워 가며 정량적으로 확인하는 도구이다. part3()가 `d_k`별로 no scale과 scaled 값을 나란히 출력하는 이유도 그 대비를 보여 주기 위해서이다.

```
def part3():
    print("Part 3. 스케일링 효과")
    print(f"  {'d_k':>6} {'no scale':>10} {'scaled':>10}   (평균 max 가중치, T=16)")
    rows = []
    for dk in (4, 64, 1024):
        a, b = softmax_peakiness(dk, False), softmax_peakiness(dk, True)
        rows.append((a, b)); print(f"  {dk:>6} {a:>10.3f} {b:>10.3f}")
    check("스케일 없으면 d_k 클수록 뾰족(>0.9)", rows[-1][0] > 0.9)
    check("스케일하면 d_k 와 거의 무관(차이<0.1)", abs(rows[-1][1] - rows[0][1]) < 0.1)
```

part3()은 "왜 $\sqrt{d_k}$로 나누는가"를 `d_k`를 바꿔 가며 실험으로 확인하는 함수이다. 앞서 설명한 softmax_peakiness를 `d_k` 값별로 호출하고, 결과를 표로 출력한 뒤 두 가지를 검증한다.

동작

```
for dk in (4, 64, 1024):
    a = softmax_peakiness(dk, False)   # 스케일 없음
    b = softmax_peakiness(dk, True)    # √d_k로 나눔
    rows.append((a, b)); print(...)
```

- `d_k`를 4, 64, 1024로 키웁니다. 작은 모델, 보통 모델(원 논문에서 쓰인 규모), 매우 큰 경우를 대표한다.
- 각 `d_k`마다 스케일 유무 두 가지의 "평균 최대 가중치"를 구해 `rows`에 저장하고 한 줄씩 출력합니다.

- `T=16`이므로 완전히 평평하면 1/16 = 0.0625, one-hot이면 1입니다.

출력 형태 (예상)

```
 d_k   no scale     scaled   (평균 max 가중치, T=16)
   4      0.3xx      0.2xx
  64      0.7xx      0.2xx
1024      0.9xx      0.2xx
```

실제 숫자는 난수에 따라 조금 달라진다. 경향은 이렇다.

- no scale: `d_k`가 커질수록 1에 가까워집니다.
- scaled: `d_k`와 상관없이 거의 일정합니다.

두 개의 check

1. rows[-1][0] > 0.9

- rows[-1]은 마지막 줄(d_k=1024)이고, [0]은 no scale 값이다.
- 스케일이 없으면 d_k=1024에서 평균 최대 가중치가 0.9를 넘는다는 확인이다. 점수의 표준편차가 √1024 = 32 정도라 softmax가 거의 one-hot이 된다.

2. abs(rows[-1][1] - rows[0][1]) < 0.1

- [1]은 scaled 값이다. d_k=1024일 때와 d_k=4일 때의 차이가 0.1 미만이라는 확인이다.
- √d_k로 나누면 점수 분산이 d_k와 무관하게 약 1로 유지된다. 그래서 softmax의 뾰족함도 d_k에 거의 영향받지 않는다.

정리

	d_k 증가 시	결과
스케일 없음	점수 분산 ∝ d_k → softmax 포화	기울기 소실 위험
스케일 있음	점수 분산 ≈ 1 유지	안정적, 학습 가능

즉 $\sqrt{d_k}$는 차원이 커져도 softmax 입력의 크기를 일정하게 유지해서 기울기가 사라지지 않게 하는 보정 계수라는 것을, 이 함수가 수치로 증명한다.

```
# ------------------------------------------------------------------------
# Part 4. 마스크 (디코더의 Masked Self-Attention)
#   위치 i 는 j <= i 인 토큰만 볼 수 있다 -> 하삼각 행렬 마스크
# ------------------------------------------------------------------------
```

Transformer에서 mask는 어텐션이 볼 수 없는 위치를 막아서, 각 토큰이 참조할 수 있는 범위를 제한하는 장치다. 이 코드에서는 attention() 안의 두 줄이 핵심이다.

```
if mask is not None:
    scores = np.where(mask, scores, -1e9)   # 못 보는 위치는 -inf에 가깝게

A = softmax(scores, axis=-1)
```

동작 원리

1. mask는 (T, T) bool 행렬이고, True면 볼 수 있고 False면 막는다.
2. 막을 위치의 점수를 -1e9(사실상 -∞)로 바꾼다.
3. softmax에서 exp(-1e9) ≈ 0이 되므로, 그 위치의 어텐션 가중치가 0이 된다.
4. 남은 위치들만으로 합이 1이 되도록 다시 정규화된다.

점수를 softmax 앞에서 바꾸는 것이 중요합니다. 가중치를 계산한 뒤에 0으로 만들면 행의 합이 1이 안 되기 때문이다. 앞에서 막으면 합이 1인 상태가 유지된다.

주요 용도 두 가지

1. Causal mask (미래 가리기), 디코더의 Masked Self-Attention

위치 i는 j ≤ i인 토큰만 볼 수 있다. 하삼각 행렬 형태이다.

```
   t0 t1 t2 t3
```

t0 [1 0 0 0]
t1 [1 1 0 0]
t2 [1 1 1 0]
t3 [1 1 1 1]

- 언어 모델은 "다음 단어를 예측"하는 것이 목표이다. 학습 때 정답이 되는 미래 토큰을 보면 그냥 베끼기만 하면 되므로 학습이 의미가 없어진다.
- 추론 때는 미래 토큰이 아직 존재하지 않는다. 학습과 추론의 조건을 같게 맞추려면 학습 때도 가려야 한다.
- mask 덕분에 한 문장을 한 번에 병렬로 넣어도, 각 위치가 자기보다 앞의 정보만으로 예측하게 된다.

- **행렬의 상삼각은 뒷쪽 정보다.**

2. Padding mask (패딩 가리기)

배치 안의 문장 길이를 맞추려고 토큰을 채운다. 이 의미 없는 토큰에 어텐션이 분산되지 않도록 해당 열을 막는다. 이 코드에는 없지만 실제 구현에서는 거의 항상 쓰인다.

구조별 사용

구성 요소	mask
인코더 self-attention (BERT 등)	없음 (양방향으로 전부 봄), padding만
디코더 self-attention (GPT 등)	causal mask
인코더-디코더 cross-attention	padding mask만

정리

mask는 모델 구조를 바꾸지 않고, softmax 직전 점수만 조작해서 정보 흐름의 방향을 통제한다. 같은 attention 함수가 mask 유무만으로 양방향 모델(인코더)과 단방향 모델(디코더)이 된다.

```
def causal_mask(T):
    return np.tril(np.ones((T, T), dtype=bool))
```

- `np.ones((T, T), dtype=bool)`
  - 모든 값이 True로 채워진 $T \times T$ 행렬을 반환한다.
- `np.tril(x)`
  - 행렬에서 하삼각형만 남겨두는 함수이다. 즉, 대각선 왼쪽은 모두 False
  - 상삼각형만 남겨둘 때는 `np.triu()`을 사용

	The	solution	that	the	professor	recommended	was	brilliant
The	self	-	-	-	-	-	-	-
solution		self	-	-	-	-	-	-
that			self	-	-	-	-	-
the				self	-	-	-	-

	The	solution	that	the	professor	recommended	was	brilliant
professor					self	-	-	-
recommended						self	-	-
was							self	-
brilliant								self

즉, 자기 자신과 과거만 볼 수 있다.

```
def part4():
    print("Part 4. causal mask")
    T = 6
    X = rng.normal(size=(T, 8))
    m = causal_mask(T)
    _, A = attention(X, X, X, mask=m)
    check("미래 위치 가중치 = 0", np.allclose(np.triu(A, 1), 0))
    check("행 합 = 1", np.allclose(A.sum(1), 1))
    X2 = X.copy(); X2[-1] += 10        # 마지막(미래) 토큰만 바꿔도
    Z1, _ = attention(X, X, X, mask=m); Z2, _ = attention(X2, X2, X2, mask=m)
    check("앞쪽 위치의 출력은 미래 토큰 변화에 영향받지 않음", np.allclose(Z1[:-1],
Z2[:-1]))
```

`part4()`는 이 마스크가 **실제로 미래를 차단하는지 세 가지 성질로 검증**하는 함수다.

준비

입력 `X`는 토큰 6개, 차원 8의 랜덤 입력이다. **차원이 8개라는 것은 토큰 하나를 8개의 숫자로 표현**한다는 의미이다.

```
    T = 6
    X = rng.normal(size=(T, 8))
    m = causal_mask(T)
    _, A = attention(X, X, X, mask=m)
```

`attention(X, X, X, m)`처럼 `Q=K=V=X`로 둔다. 투영 없이 가장 단순한 self-attention이다.

목적은 마스크 동작만 보려는 것이기 때문에 이것으로 충분하다.

원래는 Wq, Wk, Wv를 만드는 코드를 생략하고 X를 그대로 넣었다. 이렇게 하면 변수가 줄어 결과를 해석하기 쉽다.

이 단순화가 의미하는 것

	Part 2 (self_attention)	Part 4(Q=K=V=X)
Q	X @ Wq	X
K	X @ Wk	X
V	X @ Wv	X
질의 / 키 역할	분리됨	같은 벡터

Q = K이므로 점수 q_i · k_j는 X의 두 행의 내적이 된다. 즉 **비슷한 토큰끼리 점수가 높아지고**, 자기 자신과의 내적(대각선)이 보통 가장 크다. Part 1의 **Q = K = np.eye(3)**에서 "대각 성분이 가장 큼"을 확인한 것과 같은 이유이다.

한계

- Q와 K가 같으니 어텐션이 **대칭적**인 유사도(점수 행렬 S가 대칭)에 묶인다. 점수 행렬이 대칭이라는 뜻일 뿐, Softmax 후 A는 행마다 정규화되므로 대칭이 아니다. 투영이 있으면 "i가 j를 찾지만 j는 i를 안 찾는" 비대칭 관계를 만들 수 있다. Part 2에서 brilliant → solution만 강하게 만든 것이 그 예이다.
- 학습할 파라미터가 없으므로 실제 모델의 동작과는 다르다.

실제 Transformer에서는

투영은 필수이다. Wq, Wk, Wv가 있어야 같은 X에서 "찾는 관점", "찾아지는 관점", "전달할 내용"을 따로 만들 수 있고, 이 세행렬이 학습되는 핵심 파라미터이기 때문이다.

정리하면, part4는 마스크라는 한 가지 기능만 떼어 검증하려고 투영을 일부러 뺀 **최소 실험**이다.

검증 1: 미래 위치 가중치 = 0

`np.allclose(np.triu(A, 1), 0)`

`np.triu(A, 1)`은 대각선 **위쪽**(대각선 제외)만 남기는 함수이다. 그 영역이 전부 0이면 어떤 토큰도 미래에 가중치를 주지 않는다는 뜻이다. `exp(-1e9)`이 0이 되기 때문이다.

검증 2: 행 할 = 1

`np.allclose(A.sum(1), 1)`

막은 뒤에도 각 행이 확률분포로 남는지 본다. 마스크를 softmax 앞에서 적용했기 때문에 성립한다.

검증 3: 미래 토큰을 바꿔도 앞쪽 출력은 불변

```
X2 = X.copy(); X2[-1] += 10          # 마지막 토큰만 크게 변경
Z1 = attention(X,  X,  X,  mask=m)
Z2 = attention(X2, X2, X2, mask=m)
np.allclose(Z1[:-1], Z2[:-1])        # 마지막 행 제외 비교
```

이 검증은 가장 직접적이다.

마지막 토큰(t5)을 크게 바꾼 뒤 t0 ~ t4의 출력이 그대로인지 확인한다.

- t0 ~ t4에게 t5는 미래이므로 가려져 있다. 그래서 출력이 적확히 같아야 한다.
- 마지막 행(t5)은 자기 자신이 바뀌었으니 달라져도 정상이어서 [:-1]로 제외한다.

이 성질은 곧 **정보 누출(leakage) 없음**의 증명이다. 학습 때 문장 전체를 한 번에 넣어도 각 위치의 출력은 앞쪽 토큰만으로 결정된다.

정리

검증	확인하는 것
1	가중치 행렬의 구조 (상감각이 0)
2	정규화 유지
3	값의 흐름 (미래가 과거 출력에 영향 없음)

1번은 A의 모양을 보고, 3번은 실제 출력에서 확인한다. 가중치가 0이어도 다른 경로로 정보가 셀 수 있다면 3번이 실패하므로, 둘 다 확인하는 것이 의미가 있다.

---

part 5: 위치 인코딩 (Positional Encoding)

어텐션은 순서를 모르는 구조이므로, 위치별로 고유한 sin/cos 벡터를 더해 "몇 번째인지"를 알려준다.

```
#--------------------------------------------------------------------------------
--------------------
# Part 5. 위치 인코딩 -- 어텐션은 순서를 모른다
#     PE(pos, 2i) = sin(pos / 10000^(2i/d)),  PE(pos, 2i+1) = cos(pos /
10000^(2i/d))
#--------------------------------------------------------------------------------
--------------------
```

```
def positional_encoding(T, d):
    pos = np.arange(T)[:, None]
    i = np.arange(0, d, 2)[None, :]
    angle = pos / (10000 ** (i / d))
    pe = np.zeros((T, d))
    pe[:, 0::2] = np.sin(angle)
    pe[:, 1::2] = np.cos(angle)

    return pe
```

1. 문제: 어텐션은 순서를 모른다.

"A dog bit a man."과 "A man bit a dog."은 같은 단어로 이루어져 있지만 뜻은 정반대이다. 순서가 의미를 정하기 때문이다.

그런데 self-attention은 **단어를 순서 없이 한 주머니에 담은 것처럼** 다룬다. 정수 q_i · k_j는 두 토큰의 내용만으로 정해지고, "몇 번째 위치인가"는 어디에도 들어가지 않는다. 그래서 순서 정보를 따로 넣어 줘야 한다. 그 방법이 위치 인코딩이다.

아이디어는 간단하다. 위치마다 고유한 벡터(PE)를 만들어서 토큰 임베딩에 더한다.

> 입력 = 단어 임베딩 + 위치 벡터

> **Tip**
>
> **pos = np.arange(16)[:, None]**
>
> NumPy에서 자주 쓰이는 **차원 추가(reshape)** 기법이다.

1. np.arange(16)

먼저 0부터 15까지 정수를 생성한다.

결과:

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

현재 길이 16인 1차원 배열이다.

2. [:, None]

None은 새 축(axis)을 추가하라는 의미이다.

[[0]

[1]

...

[15]]

이제 (16, 1) 차원이 된다.

아래는 모두 같은 의미이다.

```
np.arange(16)[:, None]
np.arange(16).reshape(16, 1)
np.expand_dims(np.arange(16), axis=1)
```

따라서 pos = np.arange(16)[:, None]은

> 0 ~ 15를 가진 1차원 배열을 (16, 1) 형태의 열 벡터로 변환한다는 뜻이다.

아래 코드를 실행하면

```
i = np.arange(0, 16, 2)
print(i)
print(i[None, :])
```

다음과 같은 결과가 출력된다.

```
[ 0  2  4  6  8 10 12 14]
[[ 0  2  4  6  8 10 12 14]]
```

2. positional_encoding(T, d)

공식은 다음과 같다.

```
PE(pos, 2i) = sin(pos / 10000^(2i/d))      <- 짝수 번째 차원
PE(pos, 2i + 1) = cos(pos / 10000^(2u .d)) <- 홀수 번째 차원
```

코드를 한 줄씩 보면

```
pos = np.arange(T)[:, None]          # [[0],[1],...,[T-1]]  (T×1) 각 토큰의 위치 번호
i   = np.arange(0, d, 2)[None, :]    # [[0,2,4,...]]        (1×d/2) 짝수 차원 번호
angle = pos / (10000 ** (i / d))     # (T × d/2) 각 위치·차원쌍의 각도
pe = np.zeros((T, d))
pe[:, 0::2] = np.sin(angle)          # 짝수 열 = sin
pe[:, 1::2] = np.cos(angle)          # 홀수 열 = cos
```

- [:, None], [None, :]는 차원을 추가해서 broadcastring으로 (T × d/2) 표를 한 번에 만들기 위한 것
- 짝수 열에는 sin, 홀수 열에는 cos를 같은 각도로 넣는다. 그래서 두 개가 한 쌍으로 움직인다.

왜 sin/cos인가

시계를 생각해보면

- **차원마다 바늘 속도가 다르다.** $10000^{(i/d)}$로 나누기 때문에 앞쪽 차우너은 빨리 돌고(주기 짧음), 뒤쪽 차원은 아주 천천히 돈다.
- 시침, 분침, 초침을 함께 보면 시각이 하나로 정해지듯, **여러 속도의 sin/cos 값을 함께 보면 위치가 고유하게 정해진다.**
- 값이 항상 -1~1 사이라서 단어 임베딩을 압도하지 않는다.
- 위치가 k만큼 떨어진 두 벡터의 관계가 일정한 규칙(회전)으로 표현되어, 상대적 거리를 모델이 배우기 쉽다.
- 학습 파라미터가 없고, 학습 때 본 적 없는 긴 길이에도 같은 공식을 적용할 수 있다.

```
def part5():
    print("Part 5. positional encoding & 순서 불변성")
    T, d = 7, 16
    X = rng.normal(size=(T, d))
    Wq, Wk, Wv = (rng.normal(size=(d, d)) / np.sqrt(d) for _ in range(3))
```

```
    perm = rng.permutation(T)

    Z, _ = self_attention(X, Wq, Wk, Wv)
    Zp, _ = self_attention(X[perm], Wq, Wk, Wv)
    check("PE 없이: 입력을 섞으면 출력도 똑같이 섞일 뿐 (순서 정보 없음)", np.allclose(Z[perm], Zp))
    pe = positional_encoding(T, d)

    Z, _ = self_attention(X + pe, Wq, Wk, Wv)
    Zp, _ = self_attention(X[perm] + pe, Wq, Wk, Wv)
    check("PE 를 더하면 더 이상 단순 순서 바꿈이 아님", not np.allclose(Z[perm], Zp))
    check("PE shape / 범위", pe.shape == (T, d) and np.abs(pe).max() <= 1.0 + 1e-9)
    plt.figure(figsize=(7, 3.5)); plt.imshow(positional_encoding(50, 32).T, cmap="RdBu", aspect="auto")
    plt.xlabel("position"); plt.ylabel("dimension"); plt.colorbar(); plt.tight_layout()
    plt.savefig("part5_positional_encoding.png", dpi=120); plt.close()
    print("  -> part5_positional_encoding.png 저장")
```

준비

```
  T, d = 7, 16
   X = rng.normal(size=(T, d))
   Wq, Wk, Wv = (rng.normal(size=(d, d)) / np.sqrt(d) for _ in range(3))
   perm = rng.permutation(T)       # 0~6을 무작위로 섞은 순서
```

`perm`은 토큰 순서를 뒤섞는 데 쓰는 인덱스이다.

검증 1: PE가 없으면 순서를 구분 못한다.

```
Z, _ = self_attention(X, Wq, Wk, Wv)
Zp, _ = self_attention(X[perm], Wq, Wk, Wv)     # 입력 행을 섞음
np.allclose(Z[perm], Zp)                        # -> True여야 함
```

입력을 섞어서 넣은 결과(Zp)는 원래 결과(Z)를 **같은 방식으로 섞은 것**과 똑같다. 즉, 출력의 내용은 그대로고 줄 순서만 따라 바뀐다.

`X[perm]`은 열 순서는 그대로고 행순서만 바뀐다.

비유하면, 학생 명단을 아무렇게나 섞어 넣어도 각 학생의 점수는 그대로이고 명단 순서만 바뀌는 것과 같다. 모델이 "누가 먼저 왔는지"는 전혀 모른다. 이 성질을 순열 등변성(permutation equivariance)라고 한다.

검증 2: PE를 더하면 달라진다

```
Z  = self_attention(X + pe,       ...)
Zp = self_attention(X[perm] + pe, ...)
not np.allclose(Z[perm], Zp)           # → 달라야 함
```

핵심은 pe가 섞지 않은 채로 고정되어 있다는 점이다.

- `X[perm] + pe`: 섞인 단어 위에 원래 위치 순서대로 위치 벡터를 더한다. 이제 각 단어는 "나는 3번 자리에 있다"는 표식을 달고 들어간다.
- 같은 단어라도 어느 위치에 놓이느냐에 따라 입력 벡터가 달라지므로, 단순히 줄만 섞은 결과(Z[perm])와 일치하지 않는다.

즉 모델이 순서 정보를 얻게 되었다는 증명이다.

검증 3: shape와 범위

`pe.shape == (T, d) and np.abs(pe).max() <= 1.0`
모양이 (T, d)이고, sin/cos이므로 절댓값이 1을 넘지 않는지 확인한다.

그림 저장

`plt.imshow(positional_encoding(50, 32).T, cmap="RdB`
위치 50개, 차원 32개의 PE를 히트맵으로 그린다. 가로축이 위치, 세로축이 차원이다.

- 위쪽(앞쪽 차원)은 빨강/파랑이 빠르게 번갈아 나타난다. 주기가 짧기 때문이다.
- 아래쪽(뒤쪽 차원)은 색이 천천히 변한다.
- 이 줄무늬 패턴이 "여러 속도의 시계 바늘"을 눈으로 보여준다.

한눈에 정리

상황	결과
PE 없음	입력을 섞으면 출력도 그대로 섞일 뿐 → 순서 정보 없음
PE 있음	위치마다 다른 표식이 붙어 → 순서가 결과에 반영됨

한 줄 요약: 어텐션은 순서를 모르는 구조이므로, 위치별로 고유한 sin/cos 벡터를 더해 "몇 번째인지"를 알려 준다.

이 방식은 원 논문의 고정(sinusoidal) 방식이고, BERT·GPT 같은 모델은 위치 벡터를 학습시키는 방식(learned PE)을 쓰기도 한다. 최근에는 RoPE처럼 어텐션 내부에서 상대 위치를 반영하는 방식도 널리 쓰인다.

Part 6. PyTorch 멀티-헤드 어텐션과 인코더 블록

```
# ------------------------------------------------------------------
# Part 6. PyTorch: Multi-Head Attention + Transformer Encoder Block (그림 11.2)
#   x -> MHA -> Add&Norm -> FFN(Linear-GELU-Linear) -> Add&Norm
# ------------------------------------------------------------------
```

Part 1~5는 NumPy로 수식을 직접 구현했다. Part 6부터는 PyTorch로 실제 Transformer의 부품을 만든다. 구성은 두 클래스(MultiHeadAttention, EncoderBlock)와, 그것을 검증하는 part6()다.

```
def build_modules():
    import torch
    from torch import nn

    class MultiHeadAttention(nn.Module):
        def __init__(self, d_model, n_heads):
            super().__init__()
            assert d_model % n_heads == 0
            self.h, self.dk = n_heads, d_model // n_heads
            self.qkv = nn.Linear(d_model, 3 * d_model)
            self.out = nn.Linear(d_model, d_model)

        def forward(self, x, mask=None):              # x: (B, T, d_model)
            B, T, _ = x.shape
            q, k, v = self.qkv(x).chunk(3, dim=-1)

            # (B, T, d_model) -> (B, h, T, dk)

            # self.h: 헤드 개수
            q, k, v = (t.view(B, T, self.h, self.dk).transpose(1, 2) for t in (q, k, v))
```

```
            scores = q @ k.transpose(-2, -1) / self.dk ** 0.5      # (B, h, T, T)
            if mask is not None:
                scores = scores.masked_fill(~mask, float("-inf"))
            attn = scores.softmax(dim=-1)
            z = (attn @ v).transpose(1, 2).reshape(B, T, -1)       # 헤드 이어 붙이기 (concat)
            self.last_attn = attn.detach()
            return self.out(z)

    class EncoderBlock(nn.Module):
        def __init__(self, d_model, n_heads, d_ff):
            super().__init__()
            self.mha = MultiHeadAttention(d_model, n_heads)
            self.ln1, self.ln2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)
            self.ffn = nn.Sequential(nn.Linear(d_model, d_ff), nn.GELU(), nn.Linear(d_ff, d_model))

        def forward(self, x, mask=None):
            x = self.ln1(x + self.mha(x, mask))     # Add & Norm
            x = self.ln2(x + self.ffn(x))           # Add & Norm
            return x

    return MultiHeadAttention, EncoderBlock
```

1. MultiHeadAttention

왜 "멀티 헤드"인가?

지금까지의 어텐션은 하나의 관점(Wq, Wk, Wv 한 세트)으로만 토큰 관계를 봤다. 하지만 문장에는 여러 관계가 동시에 있다. "주어-동사", "형용사-명사", "대명사-지시 대상" 같은 것이다.

멀티 헤드는 **어텐션을 h개 병렬로 돌려서 각각 다른 관계를 보게 하는 것**이다. 각 헤드는 자기만의 Wq, Wk, Wv를 가지므로 서로 다른 패턴을 학습한다.

`__init__`

```
self.h, self.dk = n_heads, d_model // n_heads
self.qkv = nn.Linear(d_model, 3 * d_model)
self.out = nn.Linear(d_model, d_model)
```

- `d_model=32, n_heads=4`이면 `dk = 8`이다. 전체 차원 32를 4개 헤드가 8씩 나눠 가진다. 그래서 `d_model % n_heads == 0`을 `assert`으로 확인한다.
- `self.qkv`: `Wq`, `Wk`, `Wv` 세 개를 하나의 큰 선형층으로 합친 것이다. 출력 길이가 3×d_model이고, 나중에 3등분한다. 행렬곱 한 번으로 끝나서 더 효율적이다.
- `self.out`: 헤드들을 이어 붙인 뒤 한 번 더 섞어 주는 출력 투영(`w_0`)이다.
- NumPy 버전과 달리 `nn.Linear`는 bias도 갖고, 가중치는 학습으로 갱신된다.
- `nn.Linear()`은 PyTorch에서 **완전연결층(Fully Connected Layer)** 또는 **선형 변환(Linear Transformation)**을 구현한 모듈이다.
    - 수학적으로는 다음 연산을 수행한다.

$$y = xW^T + b$$
$$\text{or}$$
$$y = Wx + b$$

    - 여기서
        - x: 입력 벡터
        - W: 가중치(weight)
        - b: 편향(bias)
        - y: 출력

forward 한 줄씩

입력 `x`의 shape은 (`B`, `T`, `d_model`)이다. `B`는 배치(문장 개수), `T`는 토큰 수이다.

```
q, k, v = self.qkv(x).chunk(3, dim=-1)
```

`(B, T, 96)을 마지막 축에서 3등분하여 q, k, v` 각각 (B, T, 32)를 얻는다.

```
q, k, v = (t.view(B, T, self.h, self.dk).transpose(1, 2) for t in (q, k, v))
```

헤드를 나누는 핵심 부분이다.

- `view(B, T, 4, 8)`: 길이 32를 4×8로 쪼갭니다. 같은 데이터를 보는 방식만 바꾼다.
- `transpose(1, 2)`: (B, T, h, dk) → (B, h, T, dk).

> **Note**

`t.view(B, T, self.h, self.dk)`

역할

텐서의 데이터를 건들지 않고 모양(shape)만 바꾸는 함수이다.

예를 들어

```
t.shape
```

가

```
(2, 4, 512)
```

일 때,

```
t.view(2, 4, 8, 64)
```

하면

```
(2, 4, 8, 64)
```

가 된다.

즉,

$$512 \\ \downarrow \\ 8 \times 64$$

로 차원을 쪼개는 것이다.

Multi-Head Attention에서는 **512**차원 하나를 **64차원 head 8개**로 나눠서 각 헤드가 독립적으로 attention을 수행하도록 만들어야 하기 때문이다.

`transpose(1, 2)`

현재 shape가

(B, T, h, dk)

즉

(2, 4, 8, 64)

이면,

`transpose(1, 2)` 는 **1번 차원(T)**과 **2번 차원(h)**을 서로 교환한다.

$$
\begin{matrix}
\text{(B, T, h, dk)} \\
\downarrow \quad \text{swap axis 1 and 2} \\
\text{(B, h, T, dk)}
\end{matrix}
$$

가 된다.

이렇게 하면 "헤드마다 독립적인 (T, dk) 어텐션"이 배치처럼 병렬로 계산된다.

```
scores = q @ k.transpose(-2, -1) / self.dk ** 0.5      # (B, h, T, T)
```

이 코드는 Part 1의 `Q @ K.T / sqrt(d_k)`와 같다. 달라진 점은 앞에 B, h 두 축이 붙었다는 것 뿐이다. 스케일링에 쓰는 값이 전체 d_model이 아니라 **헤드 하나의 dk**라는 점에 주의해야 한다.

```
if mask is not None:
    scores = scores.masked_fill(~mask, float("-inf"))
```

Part 4의 마스크와 같은 역할이다. `~mask`는 `bool` 반전이라, 볼 수 없는 위치(False)를 -inf로 채웁니다. NumPy 버전은 -1e9, 여기서는 진짜 -inf를 쓴다.

```
attn = scores.softmax(dim=-1)
z = (attn @ v).transpose(1, 2).reshape(B, T, -1)
```

- softmax로 가중치를 만들고 V와 곱합니다. (B, h, T, dk)
- `transpose(1, 2)`로 `(B, T, h, dk)`로 되돌린 뒤 reshape로 `(B, T, h·dk = 32)`로 펼친다. 이것이 **헤드 이어 붙이기(concat)** 이다.

```
self.last_attn = attn.detach()
return self.out(z)
```

- `last_attn`은 나중에 검증하거나 시각화하려고 가중치를 저장해 둔 것이다. `detach()`는 기울기 추적에서 떼어 내서 메모리를 절약한다.
- 마지막에 out으로 헤드들의 결과를 섞어 최종 출력을 만든다.

모양 변화 정리

x	B, T, 32)
qkv 후 분할	q,k,v 각 (B, T, 32)
헤드 분리	(B, 4, T, 8)
scores	(B, 4, T, T)
attn @ v	(B, 4, T, 8)
concat	(B, T, 32)
out	(B, T, 32) ← 입력과 같은 모양

2. EncoderBlock (213–223행)

그림 11.2의 인코더 한 층입이다.

```
x = self.ln1(x + self.mha(x, mask))   # Add & Norm
x = self.ln2(x + self.ffn(x))         # Add & Norm
```

구성은 두 단계이다.

단계	하는 일
MHA	토큰끼리 정보를 주고받음 (토큰 간 섞기)
FFN	각 토큰을 독립적으로 가공 (Linear → GELU → Linear)

- `FFN(Feed Forward Network)`: d_model(32) → d_ff(64)로 늘렸다가 다시 32로 표현력을 높인다. 토큰마다 같은 가중치를 따로적용하므로 토큰 사이 정보 교환은 없다.
- `Add (잔차 연결)`: `x + sublayer(x)`. 입력을 출력에 더해서 기울기가 깊은 층에서도 잘 흐르게 하고, 층이 "x에 보탤 변화량"만 학습하게 만든다.
- `Norm (LayerNorm)`: 토큰 하나의 벡터(길이 32) 안에 습을 안정화한다.

이 구조(sublayer 뒤에 Add & Norm)를 post-LN이라고 하며 원 논문의 방식입니다. 요즘 큰 모델은 sublayer 앞에 Norm을 두는 pre-LN을 많이 씁니다.

3. build_modules() 구조

클래스를 함수 안에서 정의하고 반환한다. `import torch`를 함수 안에서 하기 때문입니다. 파일 맨 위에서 import하지 않으므로, torch가 없어도 Part 1~5(NumPy)는 실행된다. docstring의 "필요: torch (Part 6~7)"와 맞는 설계이다.

```
def part6():
    print("Part 6. PyTorch 멀티-헤드 어텐션 / 인코더 블록")
    import torch
    MultiHeadAttention, EncoderBlock = build_modules()
    torch.manual_seed(0)    # 난수를 생성하는 시드(seed)를 고정하는 함수
    x = torch.randn(2, 10, 32)
    mha = MultiHeadAttention(32, 4)
    y = mha(x)
    check("MHA 출력 shape 유지 (2,10,32)", tuple(y.shape) == (2, 10, 32))
    check("어텐션 shape (B,h,T,T) 이고 행 합 = 1",
          tuple(mha.last_attn.shape) == (2, 4, 10, 10) and
torch.allclose(mha.last_attn.sum(-1), torch.ones(2, 4, 10)))
    blk = EncoderBlock(32, 4, 64)
    check("EncoderBlock 출력 shape 유지", tuple(blk(x).shape) == (2, 10, 32))
    check("LayerNorm 후 토큰별 평균 ≈ 0", blk(x).mean(-1).abs().max().item() <
1e-4)
    n = sum(p.numel() for p in blk.parameters())
    print(f"  블록 파라미터 수: {n:,}  (d_model=32, d_ff=64)")
```

4. part6() 검증 (228–243행)

```
x = torch.randn(2, 10, 32)        # 배치 2, 토큰 10, 차원 32
mha = MultiHeadAttention(32, 4)  # 헤드 4개, dk = 8
```

check	확인하는 것
MHA 출력 shape = (2, 10, 32)	입력과 같은 모양이 유지됨
last_attn shape = (2, 4, 10, 10), 행 합 = 1	헤드별 T $T \times T$ 가중치가 만들어졌고 확률분포임
EncoderBlock 출력 shape 유지	블록을 여러 층 쌓을 수 있는 조건
blk(x).mean(-1) = 0	마지막이 LayerNorm이므로 토큰별 평균이 0에 가까움

마지막으로 blk.parameters()의 원소 수를 합산해 파라미터 개수를 출력한다. 대략 계산하면 다음과 같다.

- MHA: qkv(32×96 + 96) + out(32×32 + 32) = 3,168 + 1,056 = 4,224
- FFN: (32×64 + 64) + (64×32 + 32) = 2,112 + 2,080 = 4,192
- LayerNorm 2개: 각 64 → 128

- 합계 약 8,544

핵심 요약

- 멀티 헤드는 **d_model을 h개로 쪼개 병렬 어텐션을 돌리고 다시 이어 붙이는 것**이다. 총 계산량은 단일 헤드와 비슷하면서 여러 관점을 얻는다.
- 입출력 모양이 같기 때문에 EncoderBlock을 N층 쌓을 수 있다. 이것이 Transformer의 기본 단위이다.
- 이 파일에서는 mha(x)만 호출했고 mask는 넘기지 않았으므로, 인코더처럼 양방향으로 전부 본다.

Part 7: 장거리 의존성 학습 실험

```
# ------------------------------------------------------------------------
# Part 7. 장거리 의존성 학습 실험
#   과제: 길이 T 시퀀스에서 '맨 앞 토큰'의 값을 마지막 위치에서 맞혀라.
#   -> 맨 앞과 맨 끝은 T-1 만큼 떨어져 있다. 어텐션은 직접 연결하므로 쉽게 푼다.
# ------------------------------------------------------------------------
```

```
def part7():
    print("Part 7. 장거리 의존성 과제 학습 (CPU 로 수십 초)")
    import torch
    from torch import nn
    _, Block = build_modules()
    torch.manual_seed(0)
    V, T, D = 10, 16, 32

    class Model(nn.Module):
        def __init__(self):
            super().__init__()
            self.tok = nn.Embedding(V, D)
            self.pos = nn.Embedding(T, D)                # 학습 가능한 위치 임베딩
            self.block = Block(D, 4, 64)
            self.head = nn.Linear(D, V)

```

```
        def forward(self, x):
            h = self.tok(x) + self.pos(torch.arange(x.shape[1]))
            h = self.block(h)
            return self.head(h[:, -1])                    # 마지막 위치의 표현으로 분류

    model = Model()
    opt = torch.optim.Adam(model.parameters(), lr=3e-3)
    loss_fn = nn.CrossEntropyLoss()
    losses = []
    for step in range(600):
        x = torch.randint(0, V, (64, T))
        y = x[:, 0]                                       # 정답 = 맨 앞 토큰
        loss = loss_fn(model(x), y)
        opt.zero_grad(); loss.backward(); opt.step()
        losses.append(loss.item())
        if step % 100 == 0:
            print(f"  step {step:3d}  loss {loss.item():.3f}")
    with torch.no_grad():
        x = torch.randint(0, V, (1000, T))
        acc = (model(x).argmax(-1) == x[:, 0]).float().mean().item()
        attn = model.block.mha.last_attn.mean(dim=(0, 1))[-1]   # 마지막 위치의 어텐션 (배치/헤드 평균)
    print(f"  정확도 = {acc:.3f}  (무작위 추측 = {1 / V:.2f})")
    check("정확도 > 0.9", acc > 0.9)
    print(f"  마지막 위치가 가장 주목한 위치 = {attn.argmax().item()}  (기대: 0)")
    plt.figure(figsize=(8, 3.2))
    plt.subplot(1, 2, 1); plt.plot(losses); plt.title("training loss"); plt.xlabel("step")
    plt.subplot(1, 2, 2); plt.bar(range(T), attn.numpy()); plt.title("attention of last position"); plt.xlabel("key position")
    plt.tight_layout(); plt.savefig("part7_long_range.png", dpi=120); plt.close()
    print("  -> part7_long_range.png 저장: 어텐션이 0 번 위치에 몰렸는지 확인하세요")
```

Part 6에서 만든 인코더 블록이 **실제로 학습되는지**, 그리고 어텐션이 멀리 떨어진 토큰을 직접 연결하는지를 확인하는 실험이다.

1. 과제

입력 x: [7, 3, 9, 1, 4, ..., 2] ← 길이 16, 값은 0~9 랜덤
정답 y: 7 ← 맨 앞 토큰

맨 앞 토큰이 무엇인지를 마지막 위치에서 맞히는 문제이다.

- 맨 앞(0번)과 맨 끝(15번)은 T-1 = 15칸 떨어져 있다.
- RNN은 정보를 한 칸씩 전달하므로 15단계를 거치며 정보가 희미해진다.
- 어텐션은 마지막 위치가 0번 위치를 직접 볼 수 있으므로 거리와 상관없이 풀 수 있다. 이것이 Transformer의 장거리 의존성 장점 | 다.

정답이 "맨 앞 토큰"으로 고정되어 있어 학습이 끝나면 모델이 "마지막 위치는 0번을 봐라"를 스스로 깨우쳐야 한다.

2. 모델 (259–270행)

V, T, D = 10, 16, 32 # 어휘 크기, 시퀀스 길이, 모델 차원

구성	역할
self.tok = nn.Embedding(V, D)	토큰 번호(0~9)를 길이 32 벡터로 변환
self.pos = nn.Embedding(T, D)	위치 번호(0~15)를 길이 32 벡터로 변환 (학습 가능)
self.block = Block(D, 4, 64)	Part 6의 인코더 블록 (헤드 4개, d_ff=64)
self.head = nn.Linear(D, V)	32차원 → 10개 클래스 점수

forward의 흐름

```
h = self.tok(x) + self.pos(torch.arange(x.shape[1]))   # 토큰 + 위치
h = self.block(h)                                      # 어텐션 블록
return self.head(h[:, -1])                             # 마지막 위치만 분류
```

- **위치 임베딩**: Part 5의 sin/cos 대신 nn.Embedding을 써서 위치 벡터도 학습한다. 목적은 동일하며, 순서 정보를 주는 것이다.
- **h[:, -1]**: (B, T, D)에서 각 문장의 마지막 위치 벡터만 꺼낸다. 이 벡터가 "0번 토큰 정보를 가져왔다면" 맞힐 수 있다.
- mask는 안 넘긴다. 마지막 위치가 앞쪽을 보는 것은 허용되고, 어차피 인코더 방식이기 때문이다.

3. 학습 루프 (272–283행)

```
opt = torch.optim.Adam(model.parameters(), lr=3e-3)
loss_fn = nn.CrossEntropyLoss()
for step in range(600):
    x = torch.randint(0, V, (64, T))     # 매번 새로 만든 랜덤 데이터
    y = x[:, 0]                          # 정답 = 맨 앞 토큰
    loss = loss_fn(model(x), y)
    opt.zero_grad(); loss.backward(); opt.step()
```

- 데이터셋을 따로 두지 않고, 매 스텝마다 새 랜덤 배치 64개를 만든다. 그래서 외우기(과적합)가 불가능하고, 규칙 자체를 배워야 한다.
- CrossEntropyLoss는 10개 클래스 분류용 손실이다.
- 기본 순서는 zero_grad → backward → step | 다. 기울기 초기화, 기울기 계산, 가중치 갱신이다.
- 무작위로 찍으면 손실이 ln(10) ≈ 2.30이지만, 학습이 되면 0에 가깝게 떨어진다. 100스텝마다 손실을 출력하는 이유는 이 하락을 확인하기 위해서다.

4.평가 (284–290행)

```
with torch.no_grad():
    x = torch.randint(0, V, (1000, T))
    acc = (model(x).argmax(-1) == x[:, 0]).float().mean().item()
    attn = model.block.mha.last_attn.mean(dim=(0, 1))[-1]
```

- no_grad: 평가에는 기울기가 필요 없어 메모리와 시간을 아낀다.
- 새로운 1000개 문장으로 정확도를 계산한다. 무작위 추측은 1/10 = 0.10이고, 0.9를 넘으면 합격(check)이다.
- `last_attn`은 Part 6에서 저장해 둔 어텐션 가중치입니다. shape은 (1000, 4, 16, 16)입니다.
  - `mean(dim=(0,1))`: 배치와 헤드를 평균 → (16, 16)
  - [-1]: 마지막 질의 위치의 행 → 길이 16 벡터
- 이 벡터에서 argmax가 0이면, 마지막 위치가 맨 앞 토큰을 가장 주목했다는 뜻입니다. 모델이 올바른 방법으로 풀었다는 직접 증거입니다.

5. 그림 (291–295행)

part7_long_range.png에 두 그래프를 그린다.

- 왼쪽: 학습 손실 곡선. 2.3 근처에서 시작해 내려가는 모양이어야 한다.
- 오른쪽: 마지막 위치가 각 key 위치에 준 어텐션. 0번 막대가 높게 솟아야 ㅏㄴ다.

6.핵심 포인트

- 정확도만 보면 "맞혔다"는 것만 알 수 있고, 어텐션 막대를 보면 "왜 맞혔는지"까지 확인된다.
- 위치 정보가 꼭 필요한 과제이다. 어텐션은 순서를 모르는데, 정답은 "맨 앞"이라는 위치에 달려 있기 때문이다. 그래서 `self.pos`가 핵심 부품이고, 하단 도전 과제 B("위치 임베딩을 제거하면?")가 이 점을 묻는다. 위치 임베딩이 없으면 모델은 어느 토큰이 0번인지 구분할 수 없어 정확도가 무작위 수준(약 0.1)에 머물 것이다.

한 가지 주의

last_attn은 가장 마지막에 실행한 forward의 값을 저장한다. 평가 블록에서 `model(x)`를 호출한 직후에 읽기 때문에 평가용 1000개 문장에 대한 어텐션이 된다. 순서를 바꿔 읽으면 학습 마지막 배치의 값이 읽히니 주의한다.

```
if __name__ == "__main__":
    parts = {1: part1, 2: part2, 3: part3, 4: part4, 5: part5, 6: part6, 7: part7}
    chosen = [int(a) for a in sys.argv[1:]] or list(parts)
    for k in chosen:
        parts[k](); print()
    print("""[도전 과제]
  A. Part 2 에서 W_Q, W_K 를 바꿔 'the' 가 'professor' 를 보도록 만들어 보세요.
  B. Part 7 에서 위치 임베딩을 제거하면 어떻게 되는지 관찰하세요. 왜 그럴까요?
  C. EncoderBlock 을 2~4 층 쌓고 과제를 '맨 앞 토큰 + 맨 뒤 토큰의 합 mod 10' 으로 바꿔 보세요.
  D. causal mask 를 이용해 디코더(GPT 형) 블록을 만들고 다음 토큰 예측을 해 보세요.""
```